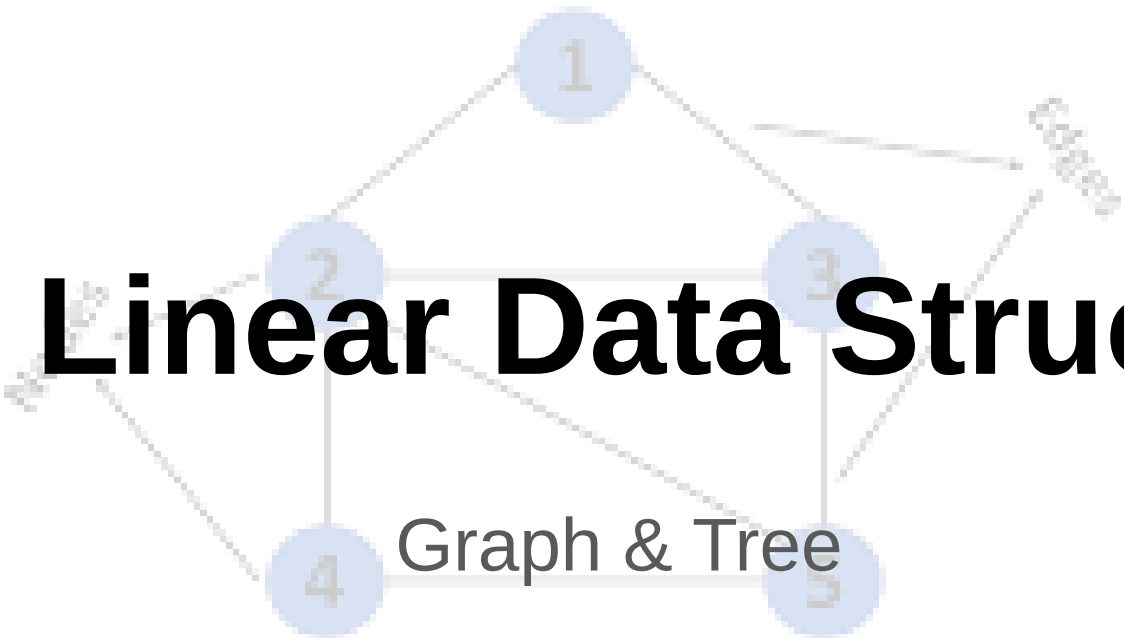




# No Linear Data Structures

**Data Structures and Algorithms with C++**

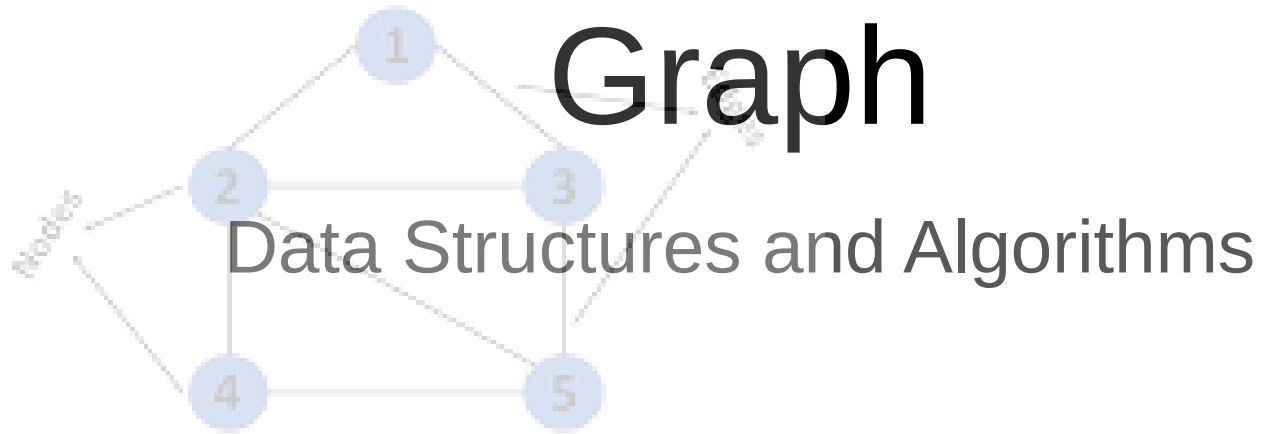


# What is a non -linear data structure?

The data structure in which the data elements are randomly arranged. The elements are non-arranged sequentially. The data elements are present at different levels. In Non-linear data structures, there are different paths for an element to reach the other element. The data elements in the non-linear data structures are connected to one or more elements. There are two types of non-linear data structures. They are:

- Graph Data Structure
- Tree Data Structure





# Introduction to Graph

A Graph is a non-linear data structure consisting of nodes and edges. The nodes are sometimes also referred to as vertices and the edges are lines or arcs that connect any two nodes in the graph. More formally a Graph can be defined as

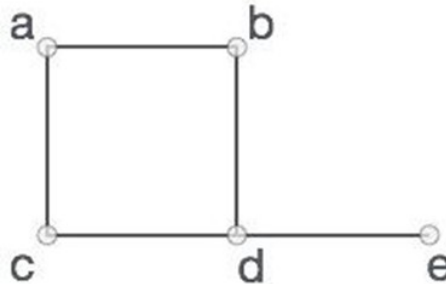
“ **consisting of a finite set of vertices(or nodes) and a set of Edges which connect a pair of nodes**”.

A graph can also be defined as :

Graph in data structures is a non-linear\_data structure made up of a finite number of nodes or vertices and the edges that connect them. Graphs are used to address real-world problems in which it represents the problem area as a network like telephone networks, circuit networks, and social networks.

For example, it can represent a single user as nodes or vertices in a telephone network, while the link between them via telephone represents edges.

Formally, a **graph** is a pair of sets  $(V, E)$ , where  $V$  is the set of vertices and  $E$  is the set of edges, connecting the pairs of vertices. Take a look at the following graph –



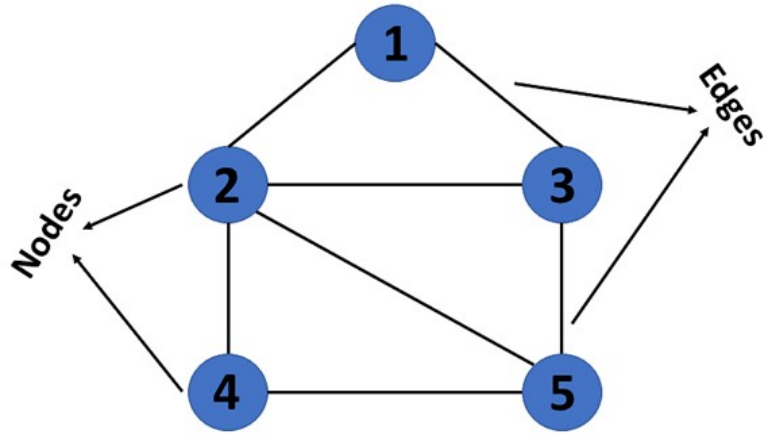
In the our graph,

$V = \{a, b, c, d, e\}$

$E = \{ab, ac, bd, cd, de\}$



## Example 2



This graph has a set of vertices

$$V = \{1, 2, 3, 4, 5\}$$

And

Set of edges

$$E = \{ (1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (4, 5) \}.$$



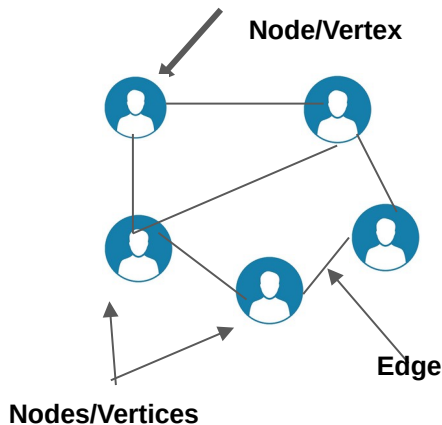
# Graph in real life

Graphs are used to address real-world problems in which it represents the problem area as a network like telephone networks, circuit networks, and social networks.

Social network Graph

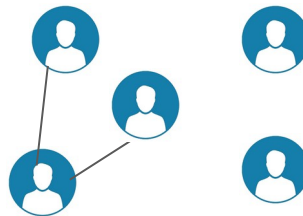
This a graph of 5 vertices and 6 edges

In case of social media, a node is compared to a user. All five nodes are users.



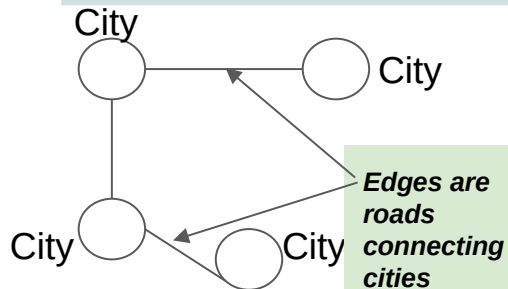
In this case of social media, an edge is compared to friendship which means two users are friends.

Here this graph can represent users of social media where one user has two friends and two users has one friend, other two users don't have a friend



This a graph of 5 vertices

Roadmap Graph

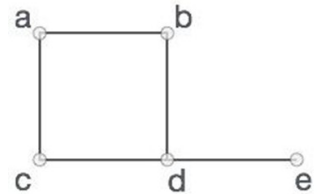


Ireland Node is not connected to other cities



# Graph terminologies

1. **An edge** is one of the two primary units used to form graphs. A line between two vertices. Each edge has two ends, which are vertices to which it is attached.
2. **Adjacent vertices:** Two vertices/nodes having a direct edge connecting them are adjacent. A and b are adjacent. The adjacent vertices are endpoints of the same edge.
3. **Degree of a Node:** The number of edges going to that node. The degree of d is 3, degree of b is 2
4. **Path:** A collection of edges from one node to another.  $a \rightarrow b \rightarrow d \rightarrow e$
5. **Size of a graph:** Total number of edges
6. **Walk:** A walk is a sequence of vertices and edges of a graph i.e. if we traverse a graph then we get a walk. Note: Vertices and Edges can be repeated,
7. **Trail:** Trail is an open walk in which no edge is repeated. Vertex can be repeated. Traversing a graph such that not an edge is repeated but vertex can be repeated and it is closed also i.e. it is a closed trail
8. Minimum number of edges in a disconnected graph is 0
9. Minimum of edges in a connected graph is  $n-1$



# Types of Graphs(work)

1. Null Graph
2. Trivial Graph
3. Non-directed Graph
4. Directed Graph
5. Connected Graph
6. Disconnected Graph
7. Connected Component
8. Regular Graph
9. Complete Graph
10. Cycle Graph
11. Cyclic Graph
12. Acyclic Graph
13. Finite Graph
14. Infinite Graph
15. Simple Graph
16. Bipartite Graph
17. Planar Graph
18. Multi Graph
19. Pseudo Graph
20. Euler Graph
21. Hamiltonian Graph
22. Tree
23. Forest
24. Weighted Graph
25. Unweighted Graph



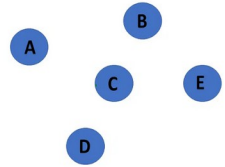


# Types of Graphs

We have many types Graphs, here we are going to discuss a few of them.

## 1. NULL Graph

The Null Graph is also known as the order zero graph. The term "**null graph**" refers to a graph with an empty edge set. In other words, a null graph has no edges, and the null graph is present with only isolated vertices in the graph. It's a reworked version of a trivial graph. If several vertices but no edges connect them, a graph  $G = (V, E)$  is a null graph.



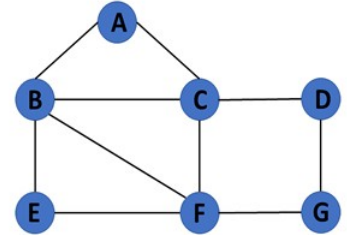
## 2. Trivial Graph

A graph  $G = (V, E)$  is trivial if it contains only a single vertex and no edges. The trivial graph is the smallest possible graph that can be created with the least number of vertices that is one vertex only.



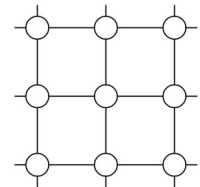
## 3. Finite Graphs

The graph  $G = (V, E)$  is called a finite graph if the number of vertices and edges in the graph is limited in number.



## 4. Infinite Graph

The graph  $G = (V, E)$  is called an infinite graph if the number of vertices and edges in the graph is interminable/infinite.



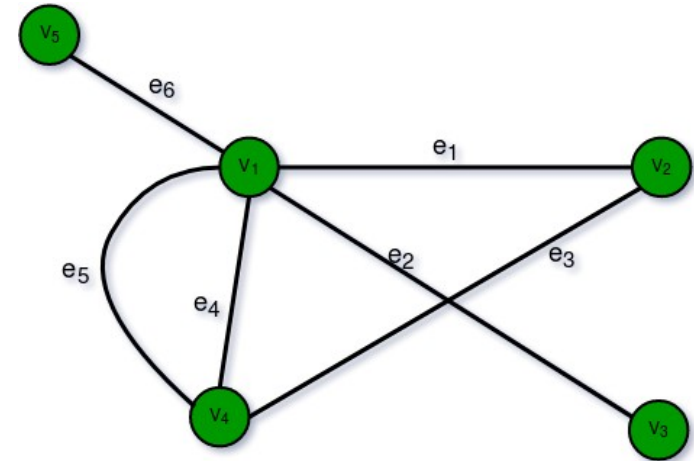
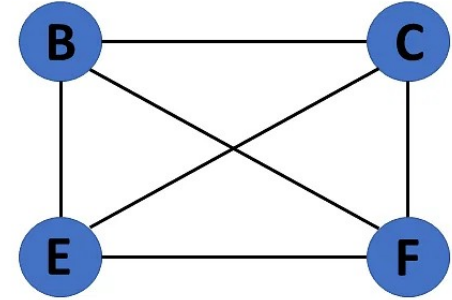
## 5. Simple Graph

If each pair of nodes or vertices in a graph  $G=(V, E)$  has only one edge, it is a simple graph. As a result, there is just one edge linking two vertices, depicting one-to-one interactions between two elements.

## 6. Multi Graph

Any graph which contains some parallel edges but doesn't contain any self-loop is called a multigraph. For example a Road Map.

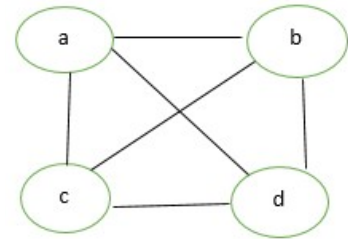
- **Parallel Edges:** If two vertices are connected with more than one edge then such edges are called parallel edges that are many routes but one destination.
- **Loop:** An edge of a graph that starts from a vertex and ends at the same vertex is called a loop or a self-loop



## 7. Complete Graph

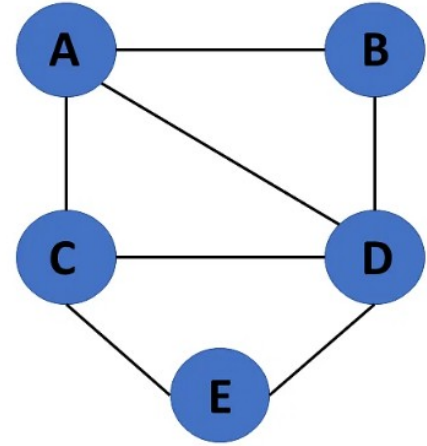
If a graph  $G = (V, E)$  is complete, it is also simple graph. But all simple graphs are not complete. Using the edges, with  $n$  number of vertices must be connected. It's also known as a full graph because each vertex's degree must be  $n-1$ ,

A graph is said to be a complete graph if, for all the vertices of the graph, there exists an edge between every pair of the vertices. In other words, we can say that all the vertices are connected to the rest of all the vertices of the graph. A complete graph of ' $n$ ' vertices contains exactly  $nC_2$  edges, and a complete graph of ' $n$ ' vertices is represented as  $K_n$ .



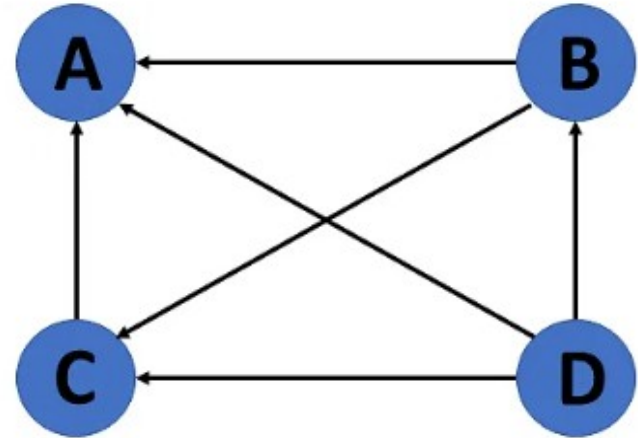
## 8. No-directed Graph or undirected Graph

- A graph in which all the edges are undirected is called as a non-directed graph.
- In other words, edges of an undirected graph do not contain any direction.



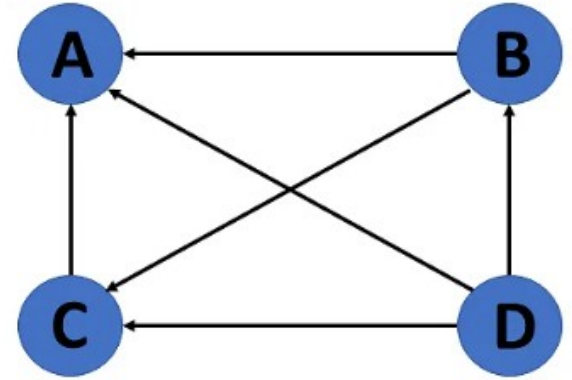
## 9. Directed Graph

- A graph in which all the edges are directed is called as a directed graph.
- In other words, all the edges of a directed graph contain some direction.
- Directed graphs are also called as **digraphs**.



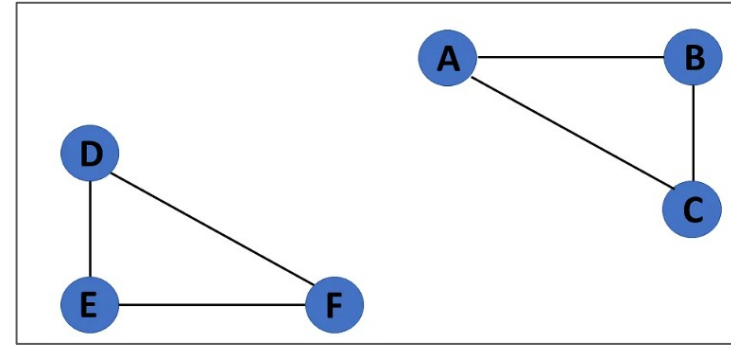
## 10. Connected Graph

- A graph in which we can visit from any one vertex to any other vertex is called as a connected graph.
- In connected graph, at least one path exists between every pair of vertices.



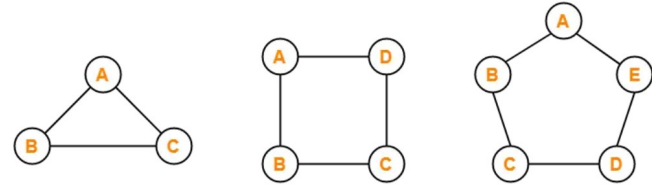
## 11. Disconnected Graph-

- A graph in which there does not exist any path between at least one pair of vertices is called as a disconnected graph.



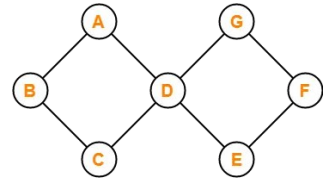
## 12. Cycle Graph

- A simple graph of 'n' vertices ( $n \geq 3$ ) and n edges forming a cycle of length 'n' is called as a cycle graph.
- In a cycle graph, all the vertices are of degree 2



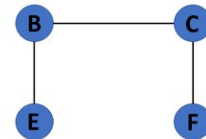
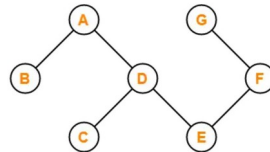
## 13. Cyclic Graph

- A graph containing at least one cycle in it is called as a cyclic graph. The graph on the right contains two cycles in it. Therefore, it is a cyclic graph.



## 14. Acyclic Graph-

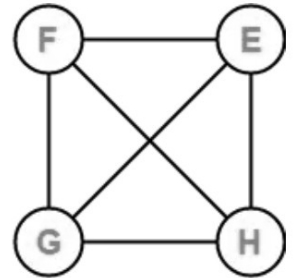
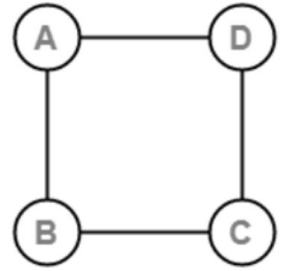
A graph not containing any cycle in it is called as an acyclic graph. See the following two graphs.



## 15. Regular Graph

For a graph to be called a regular, it should satisfy one primary condition: all graph vertices should have the same degree. By the degree of vertices, we mean the number of nodes associated with a particular vertex. If all the graph nodes have the same degree value, then the graph is called a **regular graph**. If all the vertices of a graph have the degree value of 6, then the graph is called a 6-regular graph. If all the vertices in a graph are of degree ' $k$ ', then it is called a " **$k$ -regular graph**".

**All complete graphs** are **regular** but vice versa is not possible. A regular graph is a type of undirected graph where every vertex has the same number of edges or neighbors.

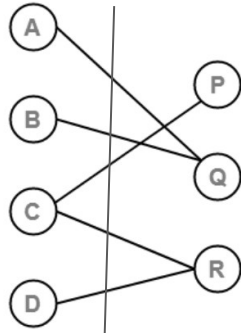




## 16. Bipartite Graph

For a graph to be a Bipartite graph, it needs to satisfy some of the basic preconditions. These conditions are:

- All the vertices of the graph should be divided into two distinct sets of vertices X and Y.
- All the vertices present in the set X should only be connected to the vertices present in the set Y with some edges. That means the vertices present in a set should not be connected to the vertex that is present in the same set.
- Both the sets that are created should be distinct that means both should not have the same vertices in t



The graph shown in the left image is divided into two vertices named set X and set Y. The contents of these sets are,

Set X = {vertex A, vertex B, vertex C, vertex D}

Set Y = {vertex P, vertex Q, vertex R}



## 17. Planar Graph

A planar graph is a graph that we can draw in a plane such that **no/without two** edges of it cross each other.

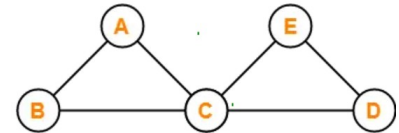
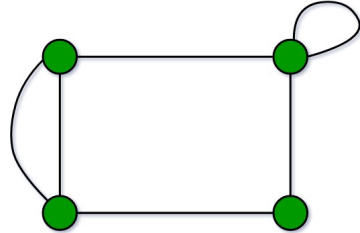
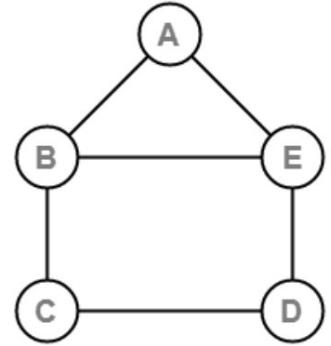
## 18. Pseudo Graph

If a graph  $G = (V, E)$  contains a self-loop besides other edges, it is a pseudograph.

A pseudograph is a type of graph that allows for the existence of loops (edges that connect a vertex to itself) and multiple edges (more than one edge connecting two vertices)

## 19. Euler Graph

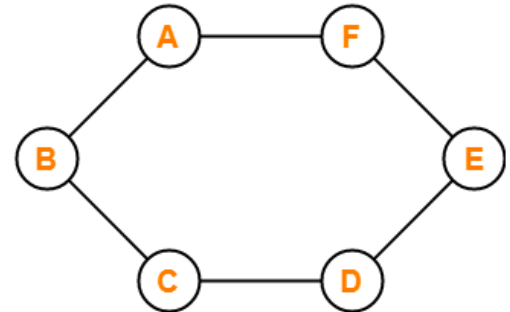
If all the vertices present in a graph have an even degree, then the graph is known as an Euler graph. By degree of a vertex, we mean the number of edges that are associated with a vertex. So for a graph to be an Euler graph, it is required that all the vertices in the graph should be associated with an even number of edges.



## 20. Hamiltonian Graph

Suppose a closed walk in the connected graph that visits every vertex of the graph exactly once (except starting vertex) without repeating the edges. Such a graph is called a Hamiltonian graph, and such a walk is called a Hamiltonian path. The Hamiltonian circuit is also known as Hamiltonian Cycle.

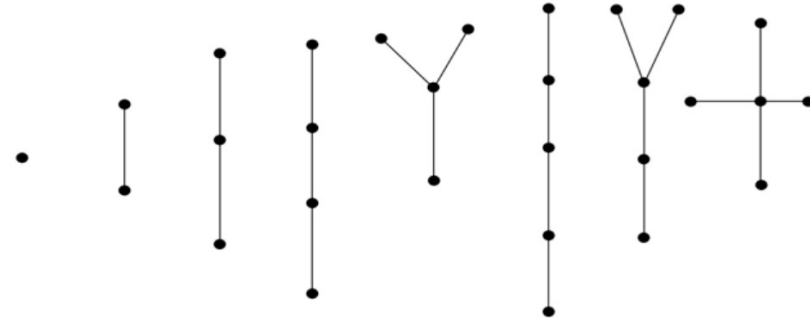
A Hamiltonian path that starts and ends at the same vertex is called a Hamiltonian circuit. Every graph that contains a Hamiltonian circuit also contains a Hamiltonian path, but vice versa is not true. There may exist more than one Hamiltonian path and Hamiltonian circuit in a graph.



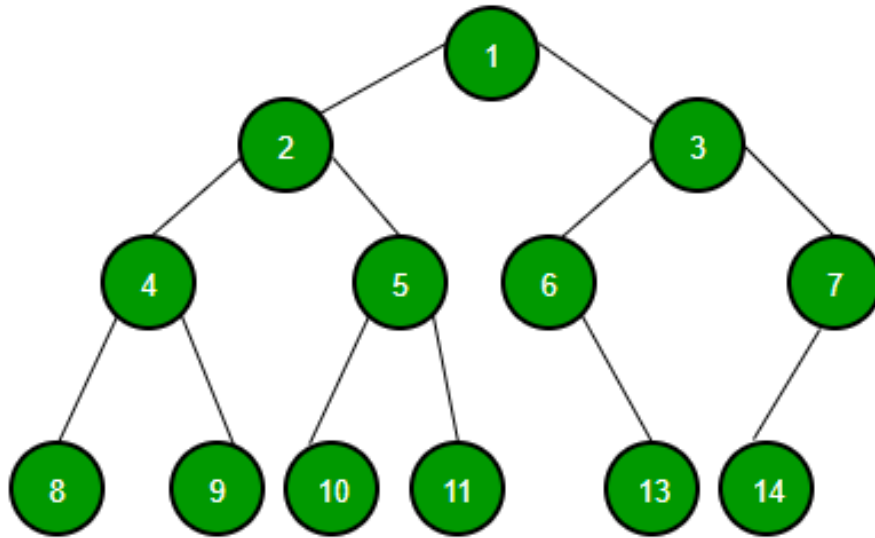
A tree is an **undirected, connected and acyclic** graph. Example: A family tree where each person is connected to their parents.

- 
- ```

graph TD
    1((1)) --- 2((2))
    1 --- 3((3))
    2 --- 4((4))
    2 --- 5((5))
    3 --- 6((6))
    3 --- 7((7))
    4 --- 8((8))
    4 --- 9((9))
    5 --- 10((10))
    5 --- 11((11))
    6 --- 13((13))
    7 --- 14((14))
  
```

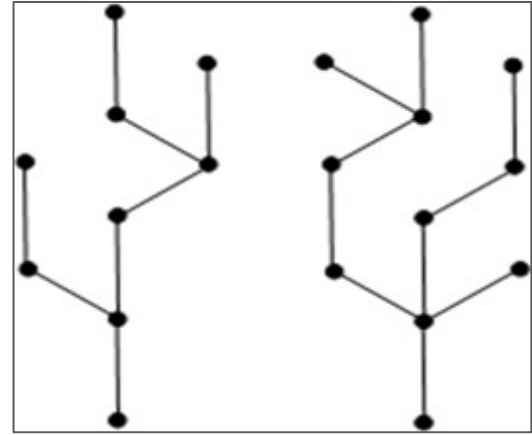


Exercise: Generate the adjacency list of the following graph



## 22. Forest

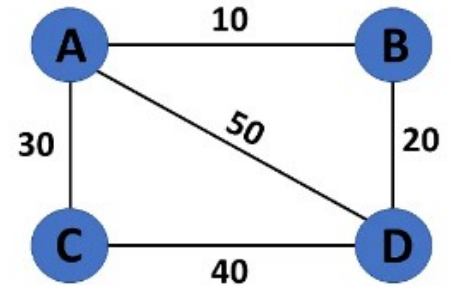
In graph theory, a **forest** is an **undirected, disconnected, acyclic graph**. In other words, a disjoint collection of trees is known as forest. Each component of a forest is tree.



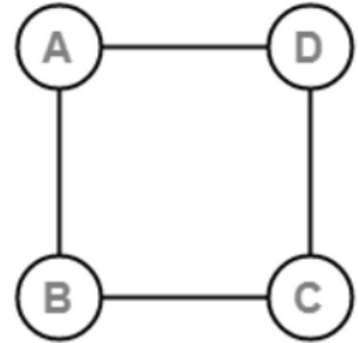
The graph on the right looks like a two sub-graphs but it is a single disconnected graph. There are no cycles in the above graph. Therefore it is a forest.



**23. Weighted Graphs:** A graph in which edges have weights or costs associated with them. A graph  $G = (V, E)$  is called a labeled or weighted graph because each edge has a value or weight representing the cost of traversing that edge. Example: A road network graph where the weights can represent the distance between two cities.



**24. Unweighted Graphs:** A graph in which edges have no weights or costs associated with them. Example: A social network graph where the edges represent friendships.



# Implementation of Graph

The graph can be presented using

1. Edge List
2. Adjacency list
3. Adjacency matrix

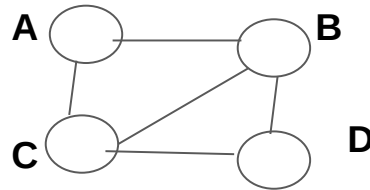




# Edge List

Two arrays to store

1. A list of vertices
2. A list of edges, an edge is a pair of vertices.



Sample Graph

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| A,B | D,C | B,D | A,C | B,C |
|-----|-----|-----|-----|-----|

# Adjacency List

Store the list of adjacent nodes with a given vertex

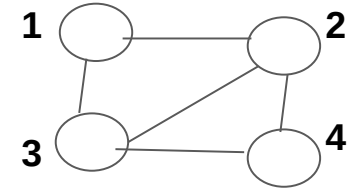
|   |  |       |
|---|--|-------|
| A |  | B,C   |
| B |  | A,C,D |
| C |  | A,B,D |
| D |  | B,C   |



# Adjacency matrix

An adjacency matrix is a square matrix of  $N \times N$  size where  $N$  is the number of nodes in the graph and it is used to represent the connections between the vertices of a graph

With adjacency matrix, if there is 4 vertices, we need create a matrix of  $4 \times 4$ .



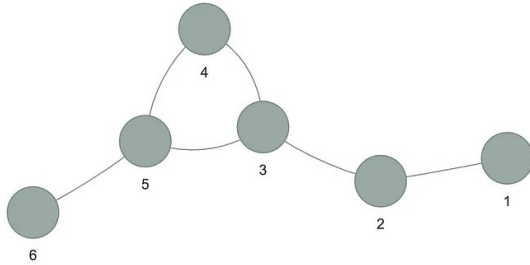
Sample Graph

|   | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | F | T | T | F |
| 2 | T | F | T | T |
| 3 | T | T | F | T |
| 4 | F | T | T | F |

**IS there an edge between (3,4) or (4,3)? If there is an edge then the value is true, otherwise the value is false.**

|   | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 0 |
| 2 | 1 | 0 | 1 | 1 |
| 3 | 1 | 1 | 0 | 1 |
| 4 | 0 | 1 | 1 | 0 |

# Undirected Graph representation



To represent this graph as the adjacency matrix  $A$ , we'll let the indices of the rows and columns represent nodes, or vertices. For the current example, we'll have 6 rows (representing nodes 1-6) and 6 columns (again, representing nodes 1-6). We should always have a square matrix! Each entry represents the presence or absence of an edge, or relationship, between the two nodes. 1 indicates an edge, 0 no edge.

$$A_{ij} = \begin{cases} 1 & \text{if there is an edge between nodes } i \text{ and } j \\ 0 & \text{otherwise} \end{cases}$$

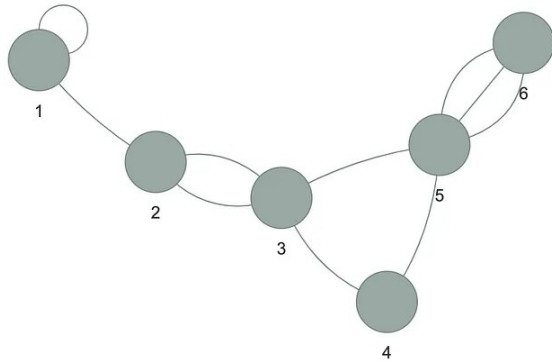
$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Since there's an edge going from node 1 to 2, we see a 1 in both  $A_{12}$  (row 1, column 2) and  $A_{21}$  (row 2, column 1). The lack of directionality in the graph results in a **symmetric matrix**. Also, notice that the diagonal consists entirely of zeros. That's because there are no edges from any node to itself. This is an easy way to check for loops!



# Representation: Undirected Graph with Loops

A Graph can have loops and nodes can even have more than one edge between them. The following example indicates a Graph with loop and multi-edges.



The Adjacency Matrix is:

$$A = \begin{bmatrix} 2 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 3 \\ 0 & 0 & 0 & 0 & 3 & 0 \end{bmatrix}$$

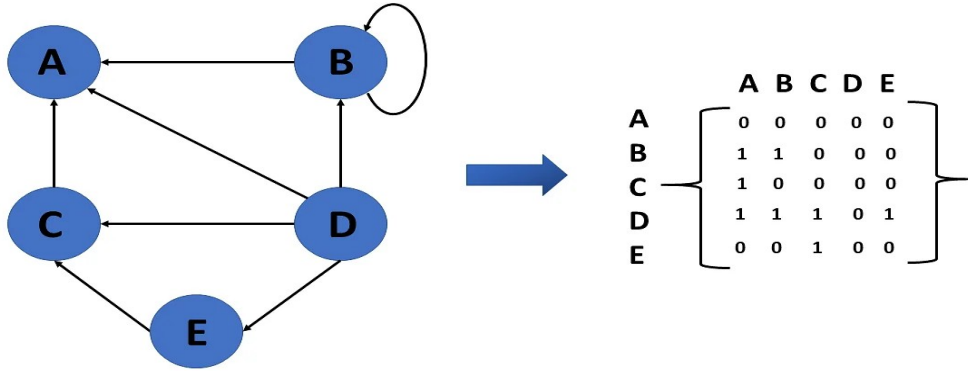
Notice that a loop is represented as a 2. For undirected graphs, each loop adds 2 since it counts each time the edge meets the node. (If there were two loops for node 1, the entry would be 4.) We can also see that there are two edges between nodes 2 and 3. Therefore,  $A_{23}$  and  $A_{32}$  are now represented by a 2. The edge number between nodes 5 and 6 has also changed accordingly.

**Note the following:**

- adjacency matrices are always square
- adjacency matrices for **undirected** graphs are symmetric
- an undirected graph with no loops will have zeros along the diagonal
- each loop in an undirected graph is represented by a 2
- adjacency matrices can account for multi-edges



# Directed Graph representation



For undirected graph. If there is an edge between two vertices;  $V_x$  to  $V_y$ ; then the value of  $A[V_x][V_y]=1$  and  $A[V_y][V_x]=1$ , otherwise the value will be zero.

For a directed graph, if there is an edge between  $V_x$  to  $V_y$ , then the value of  $A[V_x][V_y]=1$ , otherwise the value will be zero.

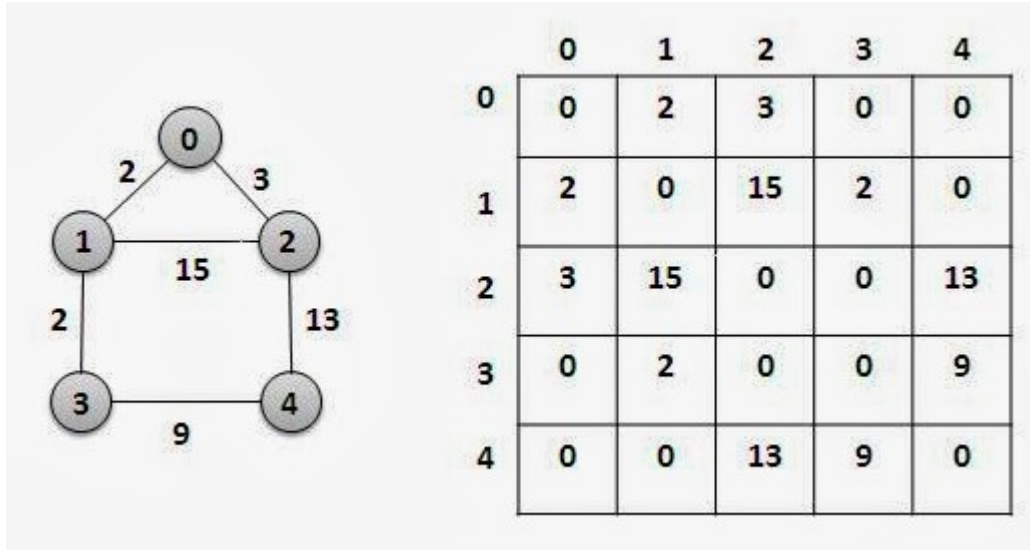
The adjacency matrix of a directed graph can be asymmetric



# Weighted Undirected Graph representation

For weighted graph, the matrix  $adj[i][j]$  is represented as:

If there is an edge between vertices  $i$  and  $j$  then  $adj[i][j] = \text{weight of the edge } (i, j)$  otherwise  $adj[i][j] = 0$ .



# Adjacency matrix : Connected undirected and unweighted Graph

```
#include <iostream>
using namespace std;
class AdjacencyMatrix
{
private:
int n;
int **adj;
public:
AdjacencyMatrix(int n)
{
/**
*Initializing an empty matrix
**/
this->n = n;
adj = new int* [n];
for (int i = 0; i < n; i++)
{
adj[i] = new int
[n];
for(int j = 0; j < n
j++)
{
adj[i][j] = 0;
}
}
display();
}
```

```
/* Adding Edge to Graph*/
void add_edge(int origin, int destin)
{
if( origin > n || destin > n || origin <= 0 || destin <= 0)
{
cout<<"Invalid edge!\n";
return;
}
adj[origin - 1][destin - 1] = 1;
}

/** Print the graph*/
void delete_edge(int origin,int destination){
if( origin > n || destination > n || origin <= 0 || destination <= 0)
{
cout<<"Invalid edge!\n";
return;
}
adj[origin-1][destination-1]=0;
}

void display(){
int i,j;
for(i = 0;i < n;i++)
{
for(j = 0; j < n; j++)
cout<<adj[i][j]<<" ";
cout<<endl;
}
};
```

```
int main(){
int nodes, max_edges, origin, destin;
cout<<"Enter number of nodes: ";
cin>>nodes;
AdjacencyMatrix am(nodes);
max_edges = nodes * nodes;
for (int i = 0; i < max_edges; i++)
{
cout<<"Edges number : "<<i+1<<endl;
cout<<"Enter edge (-1 -1 to exit): "<<endl;
cout<<"Enter Origin and destination vertices : "<<endl;
cin>>origin>>destin;
if(origin == -1 || destin == -1)
break;
am.add_edge(origin, destin);
}
am.display();

return 0;
}
```

# Adjacency matrix when a number of edges is defined

```
#include <iostream>
using namespace std;
class AdjacencyMatrix
{
private:
    int n;
    int **adj;
public:
    AdjacencyMatrix(int n)
    {
        /**
         *Initializing an empty matrix
         **/
        this->n = n;
        adj = new int* [n];
        for (int i = 0; i < n; i++)

            {
                adj[i] = new int
[n];
                for(int j = 0; j < n;
j++)
                    {
                        adj[i][j] = 0;
                    }
            }
        display();
    }
};
```

```
/* ** Adding Edge to Graph*/
void add_edge(int origin, int destin)
{
    if( origin > n || destin > n || origin <= 0
|| destin <= 0)
    {
        cout<<"Invalid edge!\n";
        return;
    }
    adj[origin - 1][destin - 1] = 1;
}

/* ** Print the graph*/
void delete_edge(int origin,int destination){
    if( origin > n || destination > n || origin <= 0
|| destination <= 0)
    {
        cout<<"Invalid edge!\n";
        Return;
    }
    adj[origin-1][destination-1]=0;
}

void display(){
    int i,j;
    for(i = 0;i < n;i++)
    {
        for(j = 0; j < n; j++)
            cout<<adj[i][j]<<" ";

        cout<<endl;
    }
}

};
```

```
int main(){
    int nodes, max_edges,edges, origin, destin;
    cout<<"Enter number of nodes: ";
    cin>>nodes;
    cout<<"Enter number of egdes: ";
    cin>>edges;
    if(edges>(nodes*nodes)){
        cout<<"invalid number of edges"<<endl;
        return 0;
    }
    AdjacencyMatrix am(nodes);
    for (int i = 0; i < edges; i++)
    {
        cout<<"Edges number : "<<i+1<<endl;
        cout<<"Enter edge (-1 -1 to exit): "<<endl;
        cout<<"Enter Origin and destination vertices
: "<<endl;

        cin>>origin>>destin;
        if(origin == -1 || destin == -1)
            break;
        am.add_edge(origin, destin);
    }
    am.display();

    am.delete_edge(1,2);
    am.delete_edge(2,2);
    cout<<"After deleting two edges"<<endl;
    am.display();

    return 0;
}
```





# Adding loops and multi edges

```
void add_edge(int origin, int destin)
{
    if( origin > n || destin > n || origin <= 0 || destin <= 0)
    {
        cout<<"Invalid edge!\n";
        return;
    }
    if(origin==destin)
        adj[origin-1][destin-1]=2;
    else
        adj[origin - 1][destin - 1]+= 1;
}
```



# Vectors

The simplest STL container is the vector, which is a data structure with contiguous blocks of memory just like an array. Because memory locations are contiguous, they can be randomly accessed so that the access time of any element of the vector is constant. Storage is managed automatically so that in an attempt to insert an element into a full vector, a larger memory block is allocated for the vector, the vector elements are copied to the new block, and the old block is released. A vector is thus a flexible array; that is, an **array whose size can be dynamically changed ie.:** the size of a vector can grow dynamically.

To use vectors, we need to include the vector header file in our program.

```
#include <vector>
```

## C++ Vector Declaration

Once we include the header file, then we can declare a vector in C++ as bellow :

```
std::vector<T> vector_name;
```

The type parameter <T> specifies the type of the vector. It can be any primitive data type such as int, char, float, etc. For example to declare a vector which dynamically store numbers we proceed as follow :

```
vector<int> num;
```

Here, num is the name of the vector and we do not specify the **size** of the vector during the declaration, because the size of a vector can grow dynamically so it is not necessary to define it.



# Vector Initialisation

## C++ Vector Initialisation

There are 3 ways to initialize a vector in C++:

### Method 1 and 2:

// list Initializer

```
vector<int> vector1 = {1, 2, 3, 4, 5};
```

// Uniform initialization

```
vector<int> vector2 {1, 2, 3, 4, 5};
```

Here, we are initializing the vector by providing values directly to the vector. Now, both vector1 and vector2 are initialized with values **1, 2, 3, 4, 5**.

### Method 3:

// constructor initialization

```
vector<int> vector3(5, 12);
```

Here, **5** is the size of the vector and **12** is the value.

This code creates an int vector with size **5** and initializes the vector with the value of **12**. So, the vector is equivalent to

```
vector<int> vector3 = {12, 12, 12, 12, 12};
```



## C++ Vector Initialization sample program

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    // method 1
    // initializer list
    vector<int> vector1 = {1, 2, 3, 4, 5};
    // method 2
    // uniform initialization
    vector<int> vector2{6, 7, 8, 9, 10};
    // method 3
    // constructor initialization
    vector<int> vector3(5, 12);
    cout << "vector from method 1 = ";
    for (int i = 0; i < vector1.size(); i++) {
        cout << vector1[i] << " ";
    }

    cout << "\n vector from method 2 = ";
    for (int i = 0; i < vector2.size(); i++) {
        cout << vector2[i] << " ";
    }

    cout << "\n vector from method 3 = ";
    for (int i = 0; i < vector3.size(); i++) {
        cout << vector3[i] << " ";
    }
    return 0;
}
```

### Output

vector from method 1 = 1 2 3 4 5

vector from method 2 = 6 7 8 9 10

vector from method 3 = 12 12 12 12 12



## C++ Vector Initialization sample program

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    // method 1
    // initializer list
    vector<int> vector1 = {1, 2, 3, 4, 5};
    // method 2
    // uniform initialization
    vector<int> vector2{6, 7, 8, 9, 10};
    // method 3
    // constructor initialization
    vector<int> vector3(5, 12);
    cout << "vector from method 1 = ";
    for (int i : vector1) {
        cout << i << " ";
    }
    cout << "\n vector from method 2 = ";
    for (int i : vector2) {
        cout << i << " ";
    }
    cout << "\n vector from method 3 = ";
    for (int i : vector3) {
        cout << i << " ";
    }
    return 0;
}
```

OR

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    // method 1
    // initializer list
    vector<int> vector1 = {1, 2, 3, 4, 5};
    // method 2
    // uniform initialization
    vector<int> vector2{6, 7, 8, 9, 10};
    // method 3
    // constructor initialization
    vector<int> vector3(5, 12);
    cout << "vector from method 1 = ";
    for (const int& i : vector1) {
        cout << i << " ";
    }
    cout << "\n vector from method 2 = ";
    for (const int& i : vector2) {
        cout << i << " ";
    }
    cout << "\n vector from method 3 = ";
    for (const int& i : vector3) {
        cout << i << " ";
    }
    return 0;
}
```

### Output

vector from method 1 = 1 2 3 4  
5

vector from method 2 = 6 7 8 9  
10

vector from method 3 = 12 12 12  
12 12



# Functions:Iterators with Vectors

1. **begin()** – Returns an iterator pointing to the first element in the vector.
2. **end()** – Returns an iterator pointing to the theoretical element that follows the last element in the vector.
3. **rbegin()** – Returns a reverse iterator pointing to the last element in the vector (reverse beginning). It moves from last to first element.
4. **rend()** – Returns a reverse iterator pointing to the theoretical element preceding the first element in the vector (considered as reverse end).
5. **cbegin()** – Returns a constant iterator pointing to the first element in the vector.
6. **chend()** – Returns a constant iterator pointing to the theoretical element that follows the last element in the vector.
7. **crbegin()** – Returns a const reverse iterator pointing to the last element in the vector (reverse beginning). It moves from last to first element.
8. **crend()** – Returns a const reverse iterator pointing to the theoretical element preceding the first element in the vector (considered as reverse end).



# Iterators in Vectors

```
// C++ program to illustrate the
// iterators in vector
#include <iostream>
#include <vector>

using namespace std;

int main()
{
    vector<int> g1;

    for (int i = 1; i <= 5; i++)
        g1.push_back(i);

    cout << "Output of begin and end: ";
    for (auto i = g1.begin(); i != g1.end(); ++i)
        cout << *i << " ";

    cout << "\nOutput of cbegin and cend: ";
    for (auto i = g1.cbegin(); i != g1.cend(); ++i)
        cout << *i << " ";

    cout << "\nOutput of rbegin and rend: ";
    for (auto ir = g1.rbegin(); ir != g1.rend(); ++ir)
        cout << *ir << " ";

    cout << "\nOutput of crbegin and crend : ";
    for (auto ir = g1.crbegin(); ir != g1.crend(); ++ir)
        cout << *ir << " ";

    return 0;
}
```

**output:**

```
Output of begin and end: 1 2 3 4 5
Output of cbegin and cend: 1 2 3 4 5
Output of rbegin and rend: 5 4 3 2 1
Output of crbegin and crend : 5 4 3 2
1
```



# Functions: Vector capacity

1. **size()** – Returns the number of elements in the vector.
2. **max\_size()** – Returns the maximum number of elements that the vector can hold.
3. **capacity()** – Returns the size of the storage space currently allocated to the vector expressed as the number of elements.
4. **resize(n)** – Resizes the container so that it contains 'n' elements.
5. **empty()** – Returns whether the container is empty.
6. **shrink\_to\_fit()** – Reduces the capacity of the container to fit its size and destroys all elements beyond the capacity.
7. **reserve()** – Requests that the vector capacity be at least enough to contain n elements.





# Sample Program with Capacity Functions

```
// C++ program to illustrate the
// capacity function in vector
#include <iostream>
#include <vector>

using namespace std;

int main()
{
    vector<int> g1;

    for (int i = 1; i <= 5; i++)
        g1.push_back(i);

    cout << "Size : " << g1.size();
    cout << "\nCapacity : " << g1.capacity();
    cout << "\nMax_Size : " << g1.max_size();

    // resizes the vector size to 4
    g1.resize(4);

    // prints the vector size after resize()
    cout << "\nSize : " << g1.size();

    // checks if the vector is empty or not
    if (g1.empty() == false)
        cout << "\nVector is not empty";
    else
        cout << "\nVector is empty";

    // Shrinks the vector
    g1.shrink_to_fit();
    cout << "\nVector elements are: ";
    for (auto it = g1.begin(); it != g1.end(); it++)
        cout << *it << " ";

    return 0;
}
```



# Functions: Access elements of the Vector

1. **reference operator [g]** – Returns a reference to the element at position 'g' in the vector
2. **at(g)** – Returns a reference to the element at position 'g' in the vector
3. **front()** – Returns a reference to the first element in the vector
4. **back()** – Returns a reference to the last element in the vector
5. **data()** – Returns a direct pointer to the memory array used internally by the vector to store its owned elements.

```
// C++ program to illustrate the
// element accessor in vector
#include <bits/stdc++.h>
using namespace std;

int main()
{
    vector<int> g1;

    for (int i = 1; i <= 10; i++)
        g1.push_back(i * 10);

    cout << "\nReference operator [g] : g1[2] = " <<
g1[2];

    cout << "\nat : g1.at(4) = " << g1.at(4);

    cout << "\nfront() : g1.front() = " << g1.front();

    cout << "\nback() : g1.back() = " << g1.back();

    // pointer to the first element
    int* pos = g1.data();

    cout << "\nThe first element is " << *pos;
    return 0;
}
```



## Functions: Modifiers on vectors

```
// C++ program to illustrate the
// Modifiers in vector
#include <bits/stdc++.h>
#include <vector>
using namespace std;

int main()
{
    // Assign vector
    vector<int> v;

    // fill the array with 10 five
    // times
    v.assign(5, 10);

    cout << "The vector elements are: ";

    for (int i = 0; i < v.size(); i++)
        cout << v[i] << " ";

    // inserts 15 to the last position
    v.push_back(15);
    int n = v.size();
    cout << "\nThe last element is: "
    << v[n - 1];

    // removes last element
    v.pop_back();

    // prints the vector
    cout << "\nThe vector elements are: ";

    for (int i = 0; i < v.size(); i++)
```

```
// inserts 5 at the beginning
v.insert(v.begin(), 5);

cout << "\nThe first element is: " << v[0];

// removes the first element
v.erase(v.begin());

cout << "\nThe first element is: " << v[0];
// inserts at the beginning
v.emplace(v.begin(), 5);
cout << "\nThe first element is: " << v[0];
// Inserts 20 at the end
v.emplace_back(20);
n = v.size();
cout << "\nThe last element is: " << v[n - 1];

// erases the vector
v.clear();
cout << "\nVector size after erase(): " <<
v.size();

// two vector to perform swap
vector<int> v1, v2;
v1.push_back(1);
v1.push_back(2);
v2.push_back(3);
v2.push_back(4);
```

```
cout << "\n\nVector 1: ";
for (int i = 0; i < v1.size(); i++)
    cout << v1[i] << " ";
cout << "\nVector 2: ";
for (int i = 0; i < v2.size(); i++)
    cout << v2[i] << " ";

// Swaps v1 and v2
v1.swap(v2);

cout << "\nAfter Swap \nVector 1: ";
for (int i = 0; i < v1.size(); i++)
    cout << v1[i] << " ";

cout << "\nVector 2: ";
for (int i = 0; i < v2.size(); i++)
    cout << v2[i] << " ";
};
}
```

# Output

The `vector` elements are: 10 10 10 10 10

The last element is: 15

The `vector` elements are: 10 10 10 10 10

The first element is: 5

The first element is: 10

The first element is: 5

The last element is: 20

Vector size after `erase()`: 0

Vector 1: 1 2

Vector 2: 3 4

After Swap

Vector 1: 3 4

Vector 2: 1 2



## Adjacency Matrix using Vectors

```
#include<iostream>
#include<vector>
using namespace std;

int main(){
    //n as number of vertices and e as number of
    edges
    int n,e;
    cin>>n>>e;
    vector<vector<int> > matrix(n,vector<int>(n,0));
    for(int i=1;i<=e;i++){
        //first vertex and second vertex
        int fv,sv;
        cin>>fv>>sv;
        matrix[fv][sv] = 1;
        matrix[sv][fv] = 1;
    }

    return 0;
```



# Graph traversal Algorithm

The process of visiting or updating each vertex in a graph is known as graph traversal. The sequence in which they visit the vertices is used to classify such traversals. Graph traversal is a subset of tree traversal.

There are two techniques to implement a graph traversal algorithm:

- Depth-first search
- Breadth-first search



# Depth First Search

In DFS we explore the nodes depth-wise. DFS starts with a root node or a start node and then explores the adjacent nodes of the current node by going deeper into the graph or a tree. This means that in DFS the nodes are explored depth-wise until a node with no children is encountered. Once the leaf node is reached, DFS backtracks and starts exploring some more nodes in a similar fashion.

In DFS we use a stack data structure for storing the nodes being explored. The edges that lead us to unexplored nodes are called 'discovery edges' while the edges leading to already visited nodes are called 'block edges'.

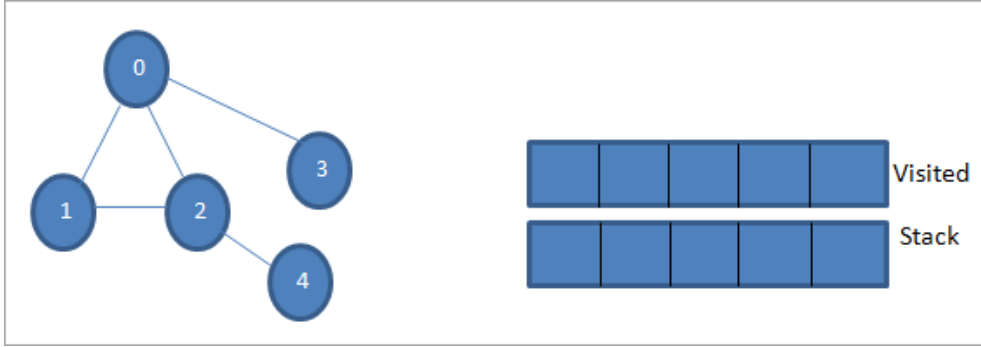
## DFS Algorithm

- **Step 1:** Insert the root node or starting node of a graph in the stack.
- **Step 2:** Pop the top item from the stack and add it to the visited list.
- **Step 3:** Find all the adjacent nodes of the node marked visited and add the ones that are not yet visited, to the stack.
- **Step 4:** Repeat steps 2 and 3 until the stack is empty.

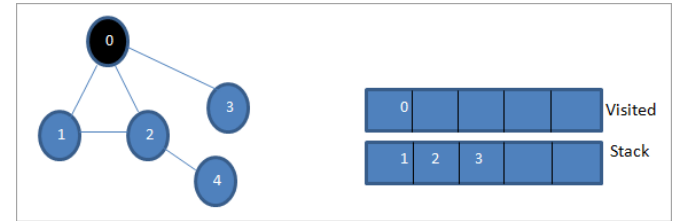


# DFS in steps

Let us now illustrate the DFS traversal of a graph.



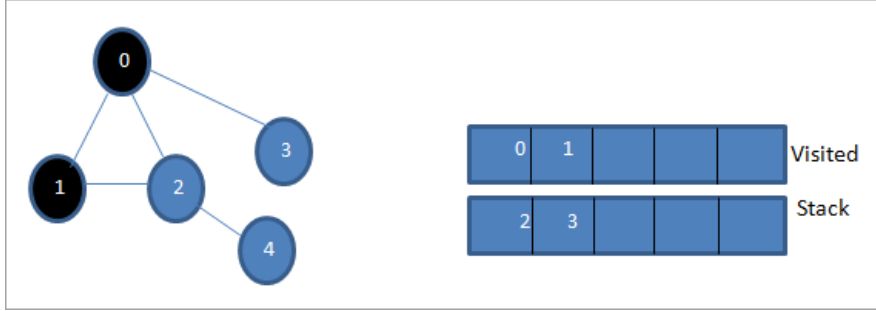
Let 0 be the starting node or source node. First, we mark it as visited and add it to the visited list. Then we push all its adjacent nodes in the stack.



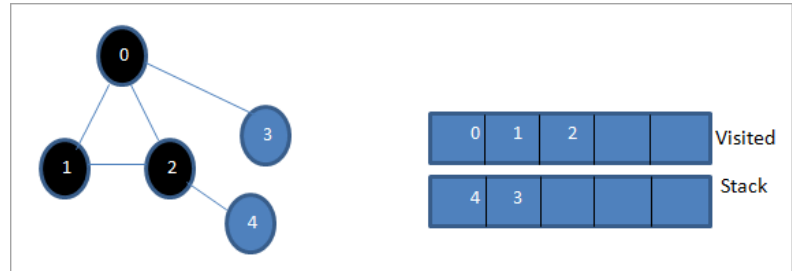


# DFS

Next, we take one of the adjacent nodes to process i.e. the top of the stack which is 1. We mark it as visited by adding it to the visited list. Now look for the adjacent nodes of 1. As 0 is already in the visited list, we ignore it and we visit 2 which is the top of the stack.

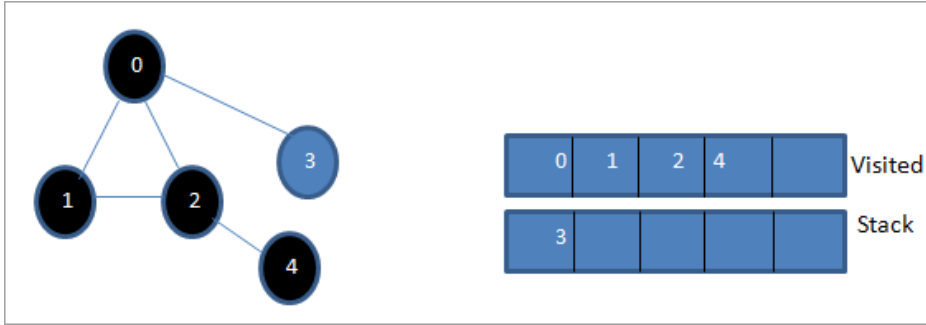


Next, we mark node 2 as visited. Its adjacent node 4 is added to the stack.

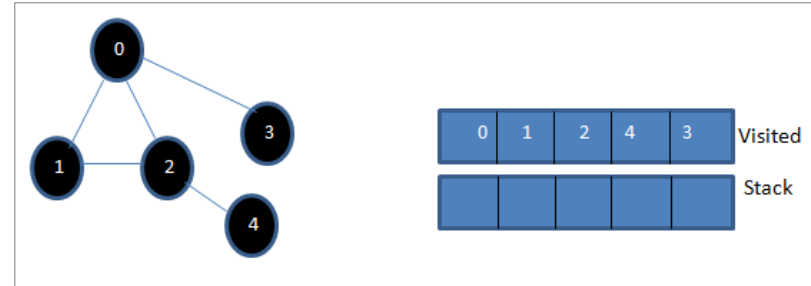


# DFS

Next, we mark 4 which is the top of the stack as visited. Node 4 has only node 2 as its adjacent which is already visited, hence we ignore it.



At this stage, only node 3 is present in the stack. Its adjacent node 0 is already visited, hence we ignore it. Now we mark 3 as visited.



Now the stack is empty and the visited list shows the sequence of the depth-first traversal of the given graph.

If we observe the given graph and the traversal sequence, we notice that for the DFS algorithm, we indeed traverse the graph depth-wise and then backtrack it again to explore new nodes.

# Recursive DFS:Function call stack.

- Create a matrix of size  $n \times n$  where every element is 0 representing there is no edge in the graph.
- Now, for every edge of the graph between the vertices  $i$  and  $j$  set  $mat[i][j] = 1$ .
- After the adjacency matrix has been created and filled, call the recursive function for the source i.e. vertex 0 that will recursively call the same function for all the vertices adjacent to it.
- Also, keep an array to keep track of the visited vertices i.e.  $visited[i] = true$  represents that vertex  $i$  has been visited before and the DFS function for some already visited node need not be called.



# DFS-Connected Graph: Recursi

```
#include<iostream>
#include<vector>
using namespace std;
void printDFS(vector<vector<int> > v, int sv, vector<bool> &visited){
    // SV as a Starting vertex
    cout<<sv<<endl;
    visited[sv] = true;
    int n = v.size();
    for(int i=0;i<n;i++){
        if(v[sv][i] == 1 && visited[i]==false){
            printDFS(v,i, visited);
        }
    }
}
```

We see that the DFS algorithm is called recursively on each vertex in the graph in order to ensure that all the vertices are visited. This implementation shown above for the DFS technique is recursive in nature and it uses a function call stack.

```
int main(){
    int n,e;
    cout<<"Enter the number of vertices and edges"<<endl;
    cin>>n>>e;
    vector<vector<int> >
matrix(n,vector<int>(n,0));
    cout<<"Reading"<< n <<"vertices numbered from 0 to "<<n-1<<endl;
    cout<<"Enter first Vertex and second vertex"<<endl;
    for(int i=1;i<=e;i++){
        int fv,sv;
        cin>>fv>>sv;
        matrix[fv][sv] = 1;
        matrix[sv][fv] = 1;
    }
    cout<<"DFS "<<endl;
    vector<bool> visited(n,false);
    printDFS(matrix,0,visited);
    return 0;
}
```



## DFS-Disconnected Graph

```
#include<iostream>
#include<vector>
using namespace std;

void printDFS(vector<vector<int> > v, int sv, vector<bool> &visited){
    cout<<sv<<endl;
    visited[sv] = true;
    int n = v.size();
    for(int i=0;i<n;i++){
        if(v[sv][i] == 1 && visited[i]==false){
            printDFS(v,i, visited);
        }
    }
}

// Consider the Disconnected Graph
void DFS(vector<vector<int> > matrix){
    int n = matrix.size();
    vector<bool> visited(n,false);
    //Count Number of Components
    int count = 0;
    for(int i=0;i<n;i++){
        if(!visited[i]){
            count++;
            printDFS(matrix,i,visited);
        }
    }
    cout<<"No of connected components is "<<count<<endl;
}
```

Whenever there is unvisited vertex, printDFS function will be called and the counter of number of components will be incremented

```
int main(){
    int n,e;
    cin>>n>>e;

    vector<vector<int> >
    matrix(n,vector<int>(n,0));
    for(int i=1;i<=e;i++){
        int fv,sv;
        cin>>fv>>sv;
        matrix[fv][sv] = 1;
        matrix[sv][fv] = 1;
    }
    cout<<"DFS "<<endl;
    DFS(matrix);
    return 0;
}
```



# Iterative: DFS Program using stack

```
#include<iostream>
#include<vector>
#include<stack>
using namespace std;

void dfs(vector<vector<int> > v, int sv, vector<bool> &visited){
    int n = v.size();
    stack<int> st;
    st.push(sv);
    while(!st.empty()){
        //cv as current vertex
        int cv = st.top();
        visited[cv] = true;
        st.pop();
        cout<<cv<<endl;
        for(int i=n-1;i>=0;i--){
            if(v[cv][i] && !visited[i]){
                st.push(i);
                // visited[i] = true;
            }
        }
    }
}
```

```
int main(){
    int n,e;
    cout<<"Enter the number of vertices and edges"<<endl;
    cin>>n>>e;

    vector<vector<int> > matrix(n,vector<int>(n,0));
    cout<<"Reading"<< n <<"vertices numbered from 0 to "<<n-1<<endl;
    cout<<"Enter first Vertex and second vertex"<<endl;
    for(int i=1;i<=e;i++){
        int fv,sv;
        cin>>fv>>sv;
        matrix[fv][sv] = 1;
        matrix[sv][fv] = 1;
    }
    vector<bool> visited(n,false);
    cout<<"DFS "<<endl;
    dfs(matrix,1,visited);
    return 0;
}
```

Note: Compared to function call stack, You may get different sequence of DFS in output because we use the stack. As the stacks follow LIFO order.



## Breadth-First Search or BFS

BFS is a search technique for finding a node in a graph data structure that meets a set of criteria.

1. It begins at the root of the graph and investigates all nodes at the current depth level before moving on to nodes at the next depth level.
2. To maintain track of the child nodes that have been encountered but not yet inspected, more memory is required, generally you use a **queue** for this purpose.

**To perform any graph traversal algorithm we have two things in mind:**

3. Each vertex has to be visited
4. Each visited vertex has to be explored

So in short the rule is visit the node then explore it.



# Algorithm of breadth-first search

**Step 1:** Consider the graph you want to navigate.

**Step 2:** Select any vertex in your graph, say  $v_1$ , from which you want to traverse the graph and use it as your root node.

**Step 3:** Examine and keep any two data structures for traversing the graph.

- Visited array will be of size of the graph size
- Queue data structure is used keep track of the current node to be visited for exploration in FIFO order

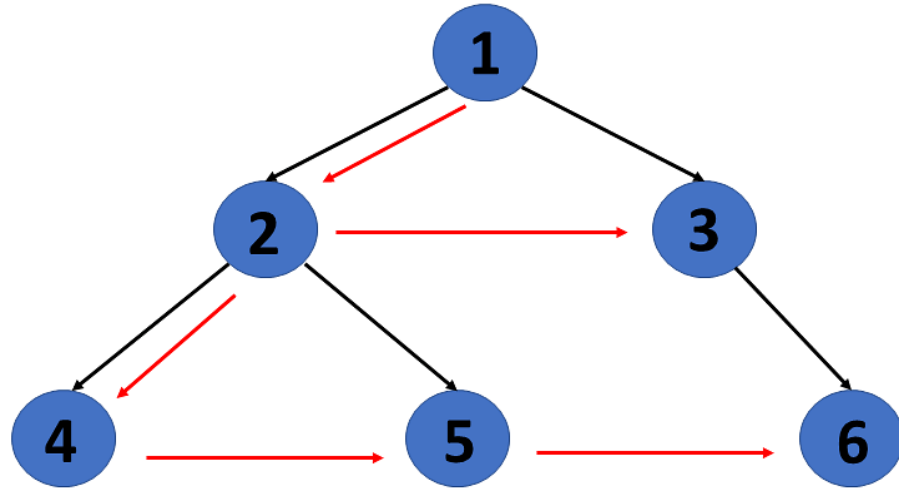
**Step 4:** Starting from the vertex  $v_1$ , you will add it to the visited array, and afterward, you will add  $v_1$ 's adjacent vertices to the queue data structure for latter visit and exploration.

**Step 5:** Now, using the FIFO concept, you must remove the current element from the queue, and put it into the visited array, and then return to the queue to add the adjacent vertices of the removed element and continue this step until no more vertices to visit.

**Step 6:** Repeat step 5 until the queue is empty and no vertex is left to be visited.







**BFS**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|



# BFS SOLUTION

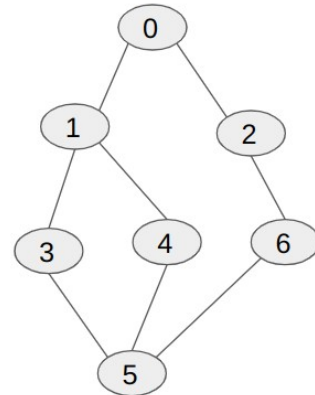
```
#include<iostream>
#include<vector>
#include<queue>
using namespace std;

void bfs(vector<vector<int> > v, int sv){
    int n = v.size();
    vector<bool> visited(n,false);
    queue<int> q;
    q.push(sv);
    visited[sv] = true;

    while(!q.empty()){
        int cv = q.front();
        q.pop();
        cout<<cv<<endl;
        for(int i=0;i<n;i++){
            if(v[cv][i] && !visited[i]){
                q.push(i);
                visited[i] = true;
            }
        }
    }
}
```

```
int main(){
    cout<<"Enter the Number of Nodes and Edges"<<endl;
    int n,e;
    cin>>n>>e;
    vector<vector<int> > matrix(n,vector<int>(n,0));
    cout<<"Enter the Edges"<<endl;
    for(int i=1;i<=e;i++){
        int fv,sv;
        cin>>fv>>sv;
        matrix[fv][sv] = 1;
        matrix[sv][fv] = 1;
    }
    cout<<"BFS"<<endl;
    bfs(matrix,0);
    return 0;
}
```

```
Enter the Number of Nodes and Edges
7
8
Enter the Edges
0 1
0 2
1 3
1 4
3 5
4 5
2 6
6 5
BFS
0
1
2
3
4
6
5
```



# BFS-Disconnected Graph

```
#include<iostream>
#include<vector>
#include<queue>
using namespace std;

void printBFS(vector<vector<int> > v, int sv, vector<bool> &visited){
    int n = v.size();
    queue<int> q;
    q.push(sv);
    visited[sv] = true;
    while(!q.empty()){
        int cv = q.front();
        q.pop();
        cout<<cv<<endl;
        for(int i=0;i<n;i++){
            if(v[cv][i] && !visited[i]){
                q.push(i);
                visited[i] = true;
            }
        }
    }
}
```

```
void BFS(vector<vector<int> > matrix){
    int n = matrix.size();
    vector<bool> visited(n,false);
    int count = 0;
    for(int i=0;i<n;i++){
        if(!visited[i]){
            count++;
            printBFS(matrix,i,visited);
        }
    }
    cout<<"No of connected components is "<<count<<endl;
}
```

```
int main(){
    int n,e;
    cin>>n>>e;
    vector<vector<int> > matrix(n,vector<int>(n,0));
    for(int i=1;i<=e;i++){
        int fv,sv;
        cin>>fv>>sv;
        matrix[fv][sv] = 1;
        matrix[sv][fv] = 1;
    }
    cout<<"BFS"<<endl;
    BFS(matrix);
    return 0;
}
```



# Weighted and directed Graphs

We have been using undirected Graphs, which means if there is an edge between  $\text{matrix}[i][j]$  there is also a edge  $\text{matrix}[j][i]$ .

In the case of directed Graphs, if there is edge  $\text{matrix}[i][j]$ ; it does not mean that there is also an edge  $\text{matrix}[j][i]$ .

When recording edges in the matrix:

$\text{matrix}[i][j]=1$ ;

For Weighted Graph, the weight or the cost of an edge will be stored instead of 1.

Eg:  $\text{matrix}[i][j]=5$ ;



# Graph Application

## 1. Computer Science

In **computer science** graph theory is used for the **study of algorithms** like:

- **Dijkstra's Algorithm**
- **Prim's Algorithm**
- **Kruskal's Algorithm**
- Graphs are used to define the **flow of computation**.
- Graphs are used to represent **networks of communication**.
- Graphs are used to represent **data organization**.
- Graph transformation systems work on rule-based in-memory manipulation of graphs. Graph databases ensure **transaction-safe, persistent storing and querying of graph structured data**.
- Graph theory is used to find **shortest path in road** or a network.
- In **Google Maps**, various locations are represented as vertices or nodes and the roads are represented as edges and graph theory is used to find the shortest path between two nodes.

## 2. Electrical Engineering

In **Electrical Engineering**, graph theory is used in **designing circuit connections**. These circuit connections are named as topologies. Some topologies are series, bridge, star and parallel topologies.



### 3. Linguistics

- In **linguistics**, graphs are mostly used for **parsing of a language tree** and **grammar of a language tree**.
- Semantic networks are used within **lexical semantics**, especially as applied to computers, modeling word meaning is easier when a given word is understood in terms of related words.
- Methods in **phonology** (e.g. theory of optimality, which uses lattice graphs) and **morphology** (e.g. morphology of finite - state, using finite-state transducers) are common in the analysis of language as a graph.

### 4. Physics and Chemistry

- In physics and chemistry, graph theory is used to **study molecules**.
- The **3D structure of complicated simulated atomic structures** can be studied quantitatively by gathering statistics on graph-theoretic properties related to the topology of the atoms.
- **Statistical physics** also uses graphs. In this field graphs can represent local connections between interacting parts of a system, as well as the dynamics of a physical process on such systems.
- Graphs are also used to express **the micro-scale channels** of porous media, in which the vertices represent the pores and the edges represent the smaller channels connecting the pores.
- Graph is also helpful in **constructing the molecular structure** as well as lattice of the molecule. It also helps us to show the bond relation between atoms and molecules, also helps in comparing structure of one molecule to another.



## 5. Computer Network

- In computer networks, the **relationships among interconnected computers** within the network follow the principles of graph theory.
- Graph theory is also used in **network security**.
- We can use the vertex coloring algorithm to find a proper **coloring of the map** with four colors.
- Vertex coloring algorithms may be used for assigning at most four different frequencies for any **GSM (Grouped Special Mobile) mobile phone networks**.

## 6. Social Sciences

- Graph theory is also used in sociology. For example, to explore rumor spreading, or to measure actors' prestige notably through the use of social network analysis software.
- Acquaintanceship and friendship graphs describe whether people know each other or not.
- In an influence graphs model, certain people can influence the behavior of others.
- In collaboration graphs model to check whether two people work together in a particular way, such as acting in a movie together.



## 7. Biology

- Nodes in biological networks represent biomolecules such as genes, proteins or metabolites, and edges connecting these nodes indicate functional, physical or chemical interactions between the corresponding biomolecules.
- Graph theory is used in transcriptional regulation networks.
- It is also used in Metabolic networks.
- In PPI (Protein - Protein interaction) networks graph theory is also useful.
- Characterizing drug - drug target relationships.

## 8. Mathematics

In mathematics, operational research is an important field. Graph theory provides many useful applications in operational research. Like:

- Minimum cost path.
- A scheduling problem.

## 9. General

Graphs are used to represent the routes between the cities/locations. With the help of a tree that is a type of graph, we can create hierarchical ordered information such as **family trees of cities and geographical locations** as nodes.





# Homework: Individual: Deadline: 06/05/2025:4pm

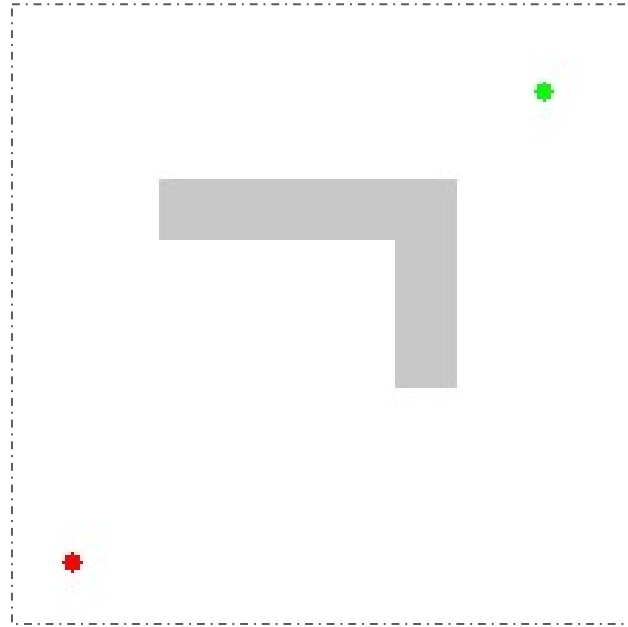
1. What is a greedy algorithm? List 5 examples
2. What is a spanning Tree?
3. What is a Minimum spanning tree (MST) of a Graph?
4. What is a shortest path in a Graph? Differentiate with MST?
5. **Kruskal Algorithm**
  - a. Describe Kruskal Algorithm using Natural Language or Pseudocode.
  - b. Draw a weighted Graph G of 6 vertices minimum, provide the MST using Kruskal Algorithm.
  - c. Using C++ and Graph theory, implement Kruskal Algorithm.
6. **Prim Algorithm:**
  - a. Describe Prim Algorithm using natural language or pseudocode
  - b. Draw a weighted Graph G of 6 vertices minimum, provide the MST using Prim Algorithm.
  - c. Using C++ and Graph Theory, implement the Prim Algorithm.
7. **Dijkstra Algorithm**
  - a. Describe Dijkstra Algorithm
  - b. Draw a weighted Graph G of 6 vertices minimum, provide the shortest path using Dijkstra Algorithm.
  - c. Using C++ and Graph theory, implement Dijkstra Algorithm.

## Note:

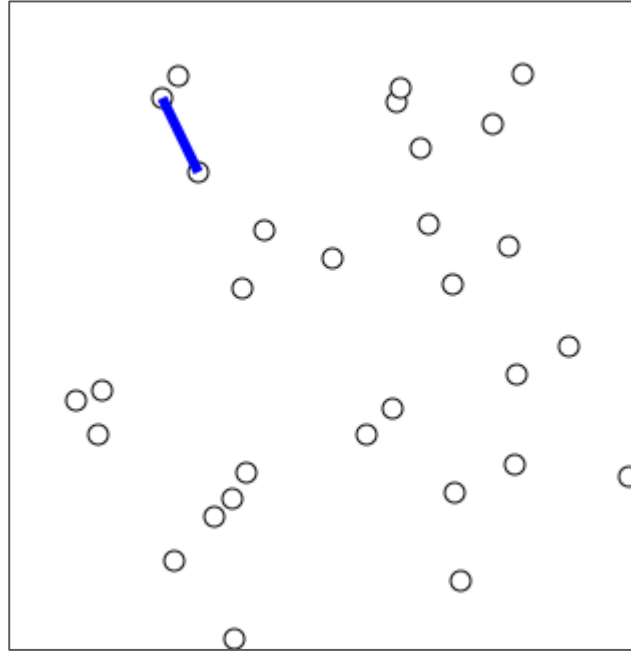
- Individual Contribution is a key.
- The best is the shortest and easiest program to drain.
- The zipped submission comprises the word document and C++ program for each implementation.
- Names must be added to the word document.



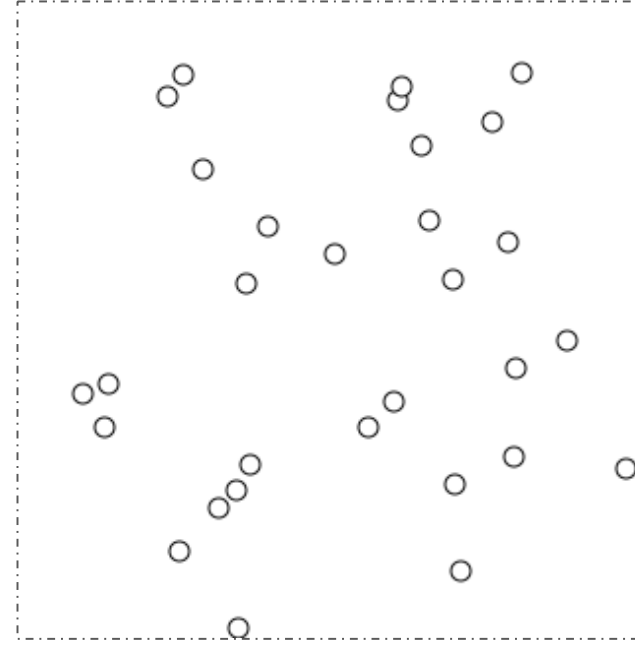
# Animation of MST Algorithms and Shortest Path Algorithm



Dijkstra  
Shortest Path



Prim  
MST



Kruskal  
MST

